

**252-0027**

**Einführung in die Programmierung  
Übungen**

**Woche 13: Probeprüfung**

**Joran Bühler  
Departement Informatik  
ETH Zürich**

# Polybox Link:



# Organisatorisches

- Nächste Woche grosses Kahoot
- -> Erste Stunde: ETA F5
- Danach: Prüfungstipps im CHN G46
- Falls noch etwas/ein Thema unklar: Mail an mich

# Probeproofung

FS 22

Frühling 2022  
252-0027 – Einführung in die Programmierung

Departement Informatik  
ETH Zürich

30. August 2022 – Schriftliche Prüfung

Nachname: \_\_\_\_\_

Vorname: \_\_\_\_\_

Legi-Nummer: \_\_\_\_\_ - \_\_\_\_\_ - \_\_\_\_\_

Sie dürfen diese Prüfung oder die Aufgaben für die Programmierprüfung erst öffnen nachdem die Aufsicht die Prüfung gestartet hat. **Wenn Sie diese Dokumente vorher öffnen gilt dies als Täuschungsversuch.**

Mit Ihrer Unterschrift bestätigen Sie, dass Sie die hier aufgeführte Person sind, Sie die Hinweise zur Kenntnis genommen haben, Sie die Aufgaben selbständig gelöst haben, Sie Ihre eigene Lösung abgeben, Sie keine Kopie der Prüfung mitnehmen, und dass Sie alle technischen Probleme, gesundheitlichen Probleme (die Ihre Leistungen in dieser Prüfung beeinträchtigten) und etwaige störende äussere Einflüsse gemeldet haben bzw. wissen, dass Sie diese melden sollen.

Unterschrift: \_\_\_\_\_

# Hinweise:

1. Öffnen Sie diese Prüfung und die Aufgabenstellung für die Programmierprüfung erst, wenn die Aufsicht den Beginn der Prüfung bekannt gibt.
2. Schreiben Sie zuerst Ihren Namen und Ihre Legi-Nummer auf die Vorderseite dieser Prüfung.
3. Im Prüfungsraum bitten wir Sie auf Gespräche zu verzichten.
4. Dieser Teil der Prüfung dauert 40 Minuten. Falls Sie sich durch irgendjemanden oder irgend etwas gestört fühlen, melden Sie dies sofort einer Aufsichtsperson.
5. Die Prüfung hat 12 Seiten. Vergewissern Sie sich dass Ihr Exemplar vollständig ist.
6. In dieser Prüfung gibt es 40 Punkte. Benutzen Sie die Anzahl der Punkte als Hinweis, wie Sie Ihre Zeit einteilen können. Fehler bei der Bearbeitung einer Aufgabe haben keinen Einfluss auf die Punkte, die Sie für andere Aufgaben erhalten. Sie können die Aufgaben in beliebiger Reihenfolge lösen.
7. Lesen Sie die Aufgabenstellungen genau durch.
8. Tragen Sie Ihre Antwort(en) direkt in die Prüfungsbögen ein. Falls Sie mehr Platz brauchen, ist Ihre Antwort wahrscheinlich zu lang.
9. Benutzen Sie einen Stift (blau oder schwarz), der nicht ausradiert werden kann. Bitte schreiben Sie deutlich und leserlich! Wenn Sie etwas durchstreichen wollen, so machen Sie dies bitte klar und eindeutig.

10. Trennen Sie nicht die zusammengeheftete Prüfung. Mit losen Blättern riskieren Sie, ein Blatt zu verlieren.
11. Es ist wichtig, dass Ihre Antworten die Aufgaben klar und *unzweideutig* behandeln. Die Klarheit der Antworten beeinflusst Ihre Note.  
Wenn Sie Annahmen (über die in den Aufgaben aufgeführten hinaus) treffen, so geben Sie diese bitte an.
12. Wenn Sie zur Toilette müssen, melden Sie dies einer Aufsichtsperson durch Handzeichen. Es darf zur gleichen Zeit immer nur eine Person zur Toilette.
13. Wir beantworten **keine inhaltlichen Fragen** während der Prüfung. Kommentare zur Aufgabe schreiben Sie bitte auf das Aufgabenblatt.
14. Wenn die Aufsicht die Prüfung beendet, schliessen Sie bitte die Prüfung und schreiben nicht mehr in die Prüfung. **Weiterarbeiten über die erlaubte Zeit gilt als Täuschungsversuch.** Bitte unterschreiben Sie die Prüfung auf der Vorderseite und legen Sie die Prüfung mit Ihrer Legitimationskarte (**"Legi"**) gut sichtbar auf Ihren Arbeitstisch.
15. Wir sammeln die Prüfung ein. Wichtig: stellen Sie unbedingt selbst sicher, dass Ihre Prüfung von einer Aufsichtsperson eingezogen wird. Stecken Sie keine Prüfung (auch keine leere) ein.

# Aufgabe 1

- 1)  $194 > 8$
- 2) 4.25
- 3) Exception



# Aufgabe 2

- `System.out.println(Arrays.toString(a[0])); // [1, 1, 3]`
- `System.out.println(Arrays.toString(a[1])); // [3, 2, 1]`
- `System.out.println(Arrays.toString(a[0])); // [1, 1, 3]`
- `System.out.println(Arrays.toString(a[1])); // [3, 2, 2]`
- `System.out.println(Arrays.toString(b[0])); // [4, 4, 6]`
- `System.out.println(Arrays.toString(b[1])); // []`

# Aufgabe 3

- $x \geq 1 \ \&\& \ x \leq (n+1) \ \&\& \ tmp = 2^{(x-1)} \ \&\& \ v == 2$

# Aufgabe 4

- Class C extends D
- Class D extends A
- Class Z
- Class A extends Z
- Class B extends A

# Aufgabe 5

1. P5 R

2. Compile-Error

3. HA

4. HA

5. G4 B

6. Exception

7. P5 R

8. P5 R

9. P5 R

# Regeln für Attribute und Methoden

- **Methoden: Dynamisch**

- Beginnt beim Dynamischen Objekt und sucht von unten nach oben
- -> Method Overriding

- **Attribute: Statisch**

- Beginnt beim Statischen Typen der Variable und sucht von unten nach oben
- -> Shadowing

# Aufgabe 6

| Ausdruck       | Gültig | Ausdruck           | Gültig |
|----------------|--------|--------------------|--------|
| $*(a+b)$       | Ja     | $(b++) += ((a+c))$ | Nein   |
| $Z = *(a + b)$ | Nein   | $A = (*b * *c)$    | Nein   |
| $Y += (z / 7)$ | Ja     | $Y++ * 7$          | Nein   |
| $X = (a = b)$  | Nein   | $K = m = n$        | Ja     |

# Aufgabe 7

- 1) WP:  $\{a > -b \ \&\& \ b > 10\}$
- 2) WP:  $\{q \geq 5\}$
- 3) WP:  $\{a > 0\}$

# Notenberechnung (HS 23)

- schriftlichen Punkte PS:
- $\text{MIN}(6, 1 + \text{MIN}((\text{PS}/20) * 3, 3 + 2 * (\text{PS} - 20) / (36 - 20)))$ 
  - (d.h. die Note 4 wurde mit 20 Punkten erreicht und die Note 6 wurde mit 36 Punkten erreicht).
- Dann wurden die Note der Prüfung berechnet als  $((2 * \text{Note\_programmieren}) + \text{Note\_schriftlich}) / 3$ .

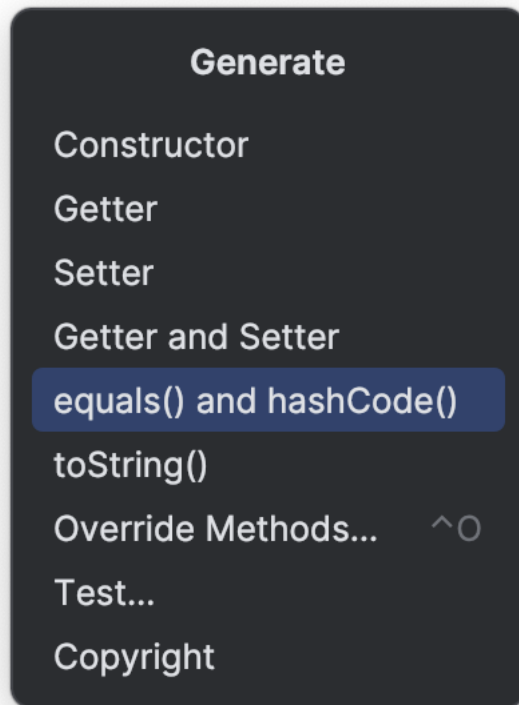


# Useful Collections

- **Am Beispiel von der Labyrinth Aufgabe**
- **Option 1: HashSet<Room>**
  - Die Methoden equals und hashCode der Objekt-Klasse überschreiben.
- **Option 2: TreeSet<Room>**
  - Comparable-Interface implementieren und compareTo-Methode überschreiben.

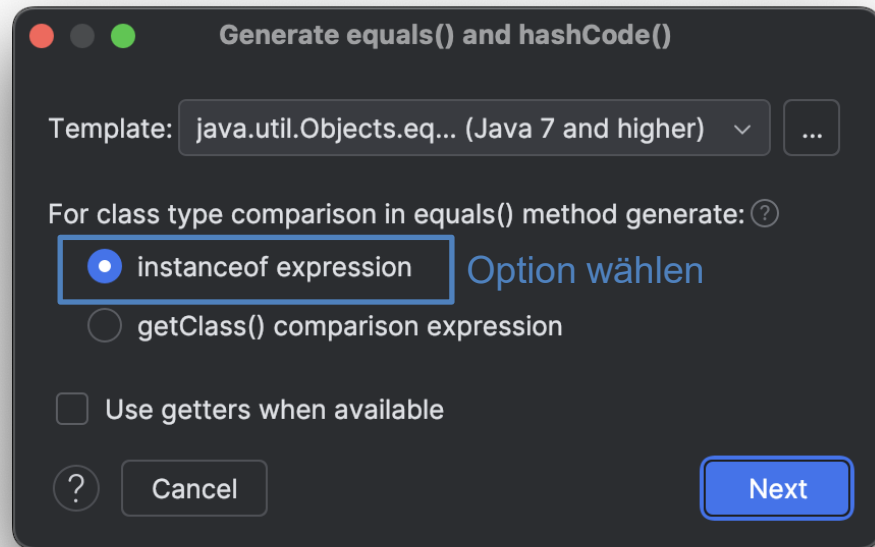
# HashSet: Methoden generieren

- HashSet<Room>
  - **In IntelliJ:** Rechtsclick -> Generate... -> Generate hashCode() und equals()

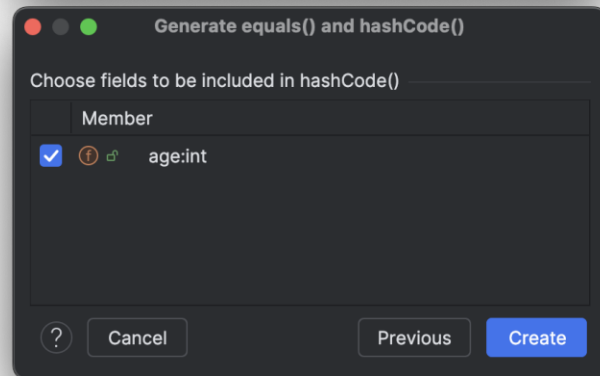
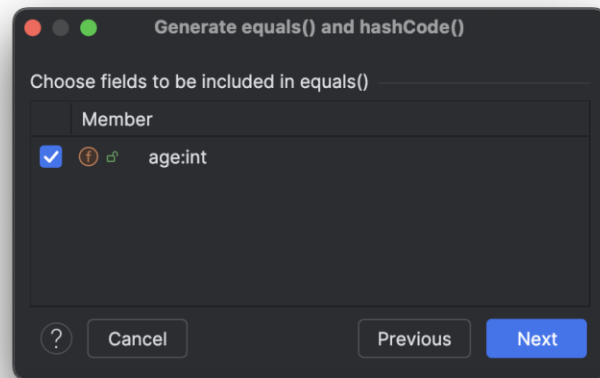


# HashSet: Methoden generieren

- `HashSet<Room>`
  - **In IntelliJ:** Rechtsclick -> Generate... -> Generate hashCode() und equals()
- `getClass()` requires exactly the same class of the other object



- `HashSet<Room>`
  - **In IntelliJ:** Rechtsclick -> Generate... -> Generate hashCode() und equals()
  - Wählt, was Sinn macht



# HashSet: Methoden generieren

```
@Override
public boolean equals(Object o) {
    if (o == null || getClass() != o.getClass()) return false;
    Room room = (Room) o;
    return age == room.age && Objects.deepEquals(doorsTo, room.doorsTo);
}

@Override
public int hashCode() {
    return Objects.hash(age, Arrays.hashCode(doorsTo));
}
```

Wir benutzen Operationen und Methoden von Superklassen um hashCode zu implementieren.



# TreeSet: Comparable Implementieren

Construction of sorted collection with non-comparable elements

```
© java.util.TreeSet<E>  
@Contract(pure = true) ↗  
public TreeSet()
```

Constructs a new, empty tree set, sorted according to the natural ordering of its elements. All elements inserted into the set must implement the `Comparable` interface. Furthermore, all such elements must be *mutually comparable*: `e1.compareTo(e2)` must not throw a `ClassCastException` for any elements `e1` and `e2` in the set. If the user attempts to add an element to the set that violates this constraint (for example, the user attempts to add a string element to a set whose elements are integers), the `add` call will throw a `ClassCastException`.

< 21 >

# TreeSet: Comparable Implementieren

- TreeSet<Room>

Damit TreeSet funktioniert muss Room das Comparable-Interface implementieren.

```
public class Room implements Comparable<Room> { 2 usages
    int age; no usages

    @Override
    public int compareTo(Room o) {
        return 0;
    }
}
```

# TreeSet: Comparable Implementieren

```
public class Room implements Comparable<Room> { 2 usages
    int age; no usages

    @Override
    public int compareTo(Room o) {
        return 0;
    }
}
```

Jetzt müssen wir nur noch die `compareTo`-Methode sinnvoll implementieren.



# TreeSet: Comparable Implementieren

- `compareTo` gibt 0 zurück, falls die Objekte `equal` sind.
- `compareTo` gibt 1 zurück, falls `this`-Objekt „grösser“ als das Parameter-Objekt ist.
- `compareTo` gibt -1 zurück, falls `this`-Objekt „kleiner“ als das Parameter-Objekt ist.

# TreeSet: Comparable Implementieren

```
@Override
public int compareTo(Room o) {
    if(this.equals(o)) {
        return 0;
    } else if(this.age > o.age) {
        return 1;
    } else {
        return -1;
    }
}
```

# Vorteile von Comparable:

- Wenn Comparable implementiert ist, dann können wir mit `Collections.sort` sortieren.
- Wenn Comparable implementiert ist, dann können wir mittels `PriorityQueue<Room>` einen Heap erstellen.
- Mit `Collections.reverseOrder()` können wir von aufsteigender Ordnung zu absteigender Ordnung wechseln.

# Vorteile von Comparable:

- Wenn Comparable implementiert ist, dann können wir mit `Collections.sort` sortieren.

```
public void sortList(List<Room> rooms) { no usages
    Collections.sort(rooms);
}
```

# Vorteile von Comparable:

- Wenn Comparable implementiert ist, dann können wir mittels `PriorityQueue<Room>` einen Heap erstellen.

```
public PriorityQueue<Room> priorityQueueFromList(List<Room> rooms) {  
    PriorityQueue<Room> priorityQueue = new PriorityQueue<>();  
    return priorityQueue;  
}
```

# Vorteile von Comparable:

- Mit `Collections.reverseOrder()` können wir von aufsteigender Ordnung zu absteigender Ordnung wechseln.

```
public void sortListReversed(List<Room> rooms) { no usages  
    Collections.sort(rooms, Collections.reverseOrder());  
}
```

# Maps

# Maps

- **Option 1: HashMap**
  - Die Methoden `equals` und `hashCode` der Objekt-Klasse überschreiben.
- **Option 2: TreeMap**
  - `Comparable`-Interface implementieren und `compareTo`-Methode überschreiben.